References

Every external source cited anywhere across 00–10, collected in one place.

The inline citations stay exactly where they are — they carry the claim, in the context that gives it meaning, and stripping them out would make the guide worse. This document serves the other purpose: it is the lookup surface (what did we cite, and where?) and the link-rot surface (which of those links has since moved or died?).

Two invariants govern the list.

Uniqueness. Every entry is a canonical URL — the address a request actually lands on after all redirects — and each appears exactly once. This is stricter than it sounds. Three counts are all true, of different moments: the original harvest of the guide found 34 raw inline URLs; after the deprecated ones were rewritten to their canonical targets, 33 raw URLs are now cited across 00–10; and those 33 resolve to just 31 unique canonical references, because two of them are aliases pointing at pages already in the list. Deduplicating the raw strings would have left duplicates hiding in plain sight — identity has to be established in the destination space, after redirects, not in the source space.

Verification. Every URL below was resolved by fetching it on 2026-07-08. The titles shown are the actual page titles, not the labels used in the inline links. Where the two differ, the inline label is noted alongside the entry, so you can still find the citation in the prose.

How to read this list

Each entry gives the page title, its URL, a one-line summary of what it covers, and CITED-IN — the numbers of the documents whose inline text cites it.

Citations are harvested from the *.machine.md roots, which are the authoritative sources in this doc set. The human twins (.md) and the PDFs are derived from those roots. That has a useful consequence: a URL appearing in a twin but not in its machine root is not a citation at all — it is a translation drift bug, and there is a check for it below.

Three documents cite no external sources at all: 01_extension_architecture, 03_agents, and 10_authoring. This is intentional rather than an oversight — they describe the toolkit's own surface, not published prior art.

URLs are the perishable part of any document. Re-run the checks in the last section before any publish or release.

Anthropic engineering (7)

- Building effective agents — https://www.anthropic.com/engineering/building-effective-agents — the workflows-versus-agents distinction; the five building-block patterns; the simplicity thesis (add machinery only when it pays). CITED-IN: 00, 05
- Effective context engineering for AI agents — https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents — context as the master finite resource; just-in-time loading; compaction; altitude (the Goldilocks system prompt). CITED-IN: 00, 08
- Equipping agents for the real world with Agent Skills — https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills — skills as progressive-disclosure capability packages, so you pay context only on demand. CITED-IN: 00, 02
- Beyond permission prompts: making Claude Code more secure and autonomous — https://www.anthropic.com/engineering/claude-code-sandboxing — sandboxing: filesystem and network isolation as the enabler of unattended runs. (Inline label: "Claude Code sandboxing" / "Sandboxing".) CITED-IN: 00, 09

- Code execution with MCP: Building more efficient agents — https://www.anthropic.com/engineering/code-execution-with-mcp — calling MCP tools through code execution to cut the per-tool token overhead. `CITED-IN: 00, 09`
- Writing effective tools for AI agents — with agents — https://www.anthropic.com/engineering/writing-tools-for-agents — tool-design principles: naming, descriptions, token economy, error surfaces. (Inline label: "Writing tools for agents".) `CITED-IN: 00, 09`
- How we built our multi-agent research system — https://www.anthropic.com/engineering/multi-agent-research-system — subagents operating in parallel, each with its own context window; orchestrator-worker in practice. (Inline label: "Multi-agent research system".) `CITED-IN: 00, 07`

The Claude blog (6)

- Lessons from building Claude Code: How we use skills — https://claude.com/blog/lessons-from-building-claude-code-how-we-use-skills — internal skill practice, and what earns a skill rather than a rule. (Inline label: "How we use Skills".) `CITED-IN: 00, 02`
- Customize Claude Code with plugins — https://claude.com/blog/claude-code-plugins — plugin packaging and distribution; the announcement post. (Inline label: "Claude Code plugins".) `CITED-IN: 00, 09`
- Steering Claude Code: when to use CLAUDE.md, skills, hooks, and subagents — https://claude.com/blog/steering-claude-code-skills-hooks-rules-subagents-and-more — how to choose among the steering surfaces. (Inline label: "Steering Claude Code".) `CITED-IN: 00`
- How Claude Code works in large codebases: Best practices and where to start — https://claude.com/blog/how-claude-code-works-in-large-codebases-best-practices-and-where-to-start — navigating big, sprawling repositories. `CITED-IN: 00`
- Getting started with loops — https://claude.com/blog/getting-started-with-loops — a loop is an agent repeating cycles of work until a stop condition is met; the four loop types. `CITED-IN: 00, 06`
- A harness for every task: dynamic workflows in Claude Code — https://claude.com/blog/a-harness-for-every-task-dynamic-workflows-in-claude-code — auto-generated JavaScript harnesses; the six orchestration patterns; the three failure modes. `CITED-IN: 00, 07`

Claude Code documentation (15)

All of these live under `https://code.claude.com/docs/en/`.

- Best practices for Claude Code — https://code.claude.com/docs/en/best-practices — explore → plan → code → commit; test-driven development; visual iteration; multi-Claude; and above all, "give Claude a check it can run." This is the most-cited source in the whole guide, with 11 inline citations. It has migrated: it used to live at `www.anthropic.com/engineering/claude-code-best-practices`, which now issues a 308 redirect to this address (see the table below). `CITED-IN: 00, 04`
- Connect Claude Code to tools via MCP — https://code.claude.com/docs/en/mcp — `CITED-IN: 00, 09`
- Create plugins — https://code.claude.com/docs/en/plugins — `CITED-IN: 00, 09`
- Run Claude Code programmatically — `https://code.claude.com/docs/en/headless` — headless `claude -p`. This page absorbs the legacy `docs.anthropic.com/en/docs/claude-code/sdk/sdk-headless` address. `CITED-IN: 00, 09`
- Agent SDK overview — https://code.claude.com/docs/en/agent-sdk/overview — the SDK library documentation. This is a distinct page from `/headless`, which covers the CLI surface; the old legacy URL conflated the two. `CITED-IN: 00, 09`
- Create custom subagents — https://code.claude.com/docs/en/sub-agents — `CITED-IN: 00`
- Extend Claude with skills — https://code.claude.com/docs/en/skills — note the alias: `/slash-commands` serves this same page and declares `/skills` as its canonical address, because custom commands were merged into skills. That is the documentary basis for the "skills are slash commands" claim in `02_skills_and_commands`. `CITED-IN: 00`
- Claude Code settings — https://code.claude.com/docs/en/settings — `CITED-IN: 00`

- Output styles — https://code.claude.com/docs/en/output-styles — CITED-IN: 00
- How Claude remembers your project — https://code.claude.com/docs/en/memory — CLAUDE.md loading and precedence. CITED-IN: 00
- Interactive mode — https://code.claude.com/docs/en/interactive-mode — CITED-IN: 00
- Hooks reference — https://code.claude.com/docs/en/hooks — CITED-IN: 00
- Claude Code GitHub Actions — https://code.claude.com/docs/en/github-actions — CITED-IN: 00
- Common workflows — https://code.claude.com/docs/en/common-workflows — CITED-IN: 00
- CLI reference — https://code.claude.com/docs/en/cli-reference — CITED-IN: 00

Model Context Protocol (2)

- What is the Model Context Protocol (MCP)? — https://modelcontextprotocol.io/docs/getting-started/intro — the protocol introduction. This is the canonical address: both the site root (modelcontextprotocol.io) and the older /introduction path resolve to this same content. CITED-IN: 00, 09
- Notion remote MCP server (endpoint) — https://mcp.notion.com/mcp — this is an MCP server endpoint, not a documentation page. It answers 401 Unauthorized, which is an authentication challenge and therefore confirms it exists. The guide cites it only as the concrete example for claude mcp add. CITED-IN: 09

Code (1)

- claude-cookbooks — agent patterns — https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents — executable notebooks for the five building-block patterns: basic_workflows.ipynb (chaining, routing, parallelization), orchestrator_workers.ipynb, evaluator_optimizer.ipynb, and async_multi_agent_orchestration.ipynb. Read the code, not just the names. (Inline labels: “agents cookbook” / “Anthropic cookbook — agent patterns”.) CITED-IN: 00, 05

How 34 harvested URLs became 31 unique references

Resolving each raw URL to its canonical target collapses three pairs and relocates one page. Deduplicating the raw strings alone would have left 34 entries, three of them duplicates.

Of the five rows below, three were deprecated or restructured and have been rewritten to their canonical targets throughout 00–10. Two of those targets are URLs new to the set; the third, /headless, was already cited by 09. That is what takes the 34 harvested URLs down to the 33 now cited. The remaining two rows are live aliases, left in place because they still resolve and read correctly in context; they are what collapse 33 raw URLs into 31 unique canonical entries.

Raw URL (as cited inline)	Resolves to	Kind
https://code.claude.com/docs/en/slash-commands	https://code.claude.com/docs/en/skills	Alias — the same page; it declares /skills canonical. Still cited in 00, where it reads correctly in context.
https://modelcontextprotocol.io	https://modelcontextprotocol.io/docs/getting-started/intro	Alias — the site root serves the introduction. Still cited in 09 as a site-root link.
https://modelcontextprotocol.io/introduction	https://modelcontextprotocol.io/docs/getting-started/intro	Alias — the page was restructured; the old path still resolves. Rewritten in 00 and 09.

Raw URL (as cited inline)	Resolves to	Kind
`https://docs.anthropic.com/en/docs/claude-code/sdk/sdk-headless`	`https://code.claude.com/docs/en/headless` (301 chain)	Redirected (301 chain) — legacy domain, deprecated. Rewritten in `00`.
`https://www.anthropic.com/engineering/claude-code-best-practices`	`https://code.claude.com/docs/en/best-practices` (308)	Migrated — the blog post was absorbed into the documentation. Rewritten in `00` and `04`, covering 11 citations.

The raw URLs in that table are written in full `https://` form deliberately. The canary below greps for `https?://`, so a raw URL recorded as a bare hostname would read as an unrecorded citation and trip the alarm.

What was done on 2026-07-08. The two deprecated sources — the legacy `sdk-headless` URL and the migrated `claude-code-best-practices` URL — together with the restructured `modelcontextprotocol.io/introduction` path, were rewritten to their canonical targets in the citing documents `00`, `04`, and `09`, in both the machine roots and the human twins, so that the inline citations and this list agree with one another.

What was deliberately left alone. The `/slash-commands` link and the bare `modelcontextprotocol.io` site-root link both still resolve and read naturally where they appear, so they were recorded here as aliases rather than rewritten.

One caveat, honestly flagged. The `sdk-headless` 301 chain's literal terminal `Location` header points at the `docs.claude.com` mirror host, which the fetch tool's domain-safety layer blocks. The identical content is confirmed live (HTTP 200) at `code.claude.com/docs/en/headless`, and that is the canonical headless page according to the current documentation index — so that is the address cited here. The mirror's own live status is inferred, not directly observed.

Keeping this list current

A references list rots silently. Nothing fails when a link dies, and nothing complains when someone adds a citation upstream without recording it here. So the check is mechanical rather than remembered.

The rule: if you add an inline citation to any of `00`–`10`, add its entry here in the same edit. The canary below is what catches you when you forget.

Re-derive the raw URL set from the authoritative machine roots:

```
cd payload/docs/advanced
grep -rhoE 'https?://[^ )"`,>]+' . --include='*.machine.md' \
  | sed 's/[.,;:]*$//' | sort -u
```

The canary. Every URL cited in `00`–`10` must appear in this document. Expect empty output; any line printed is an unrecorded citation:

```
cd payload/docs/advanced
comm -23 \
  <(grep -rhoE 'https?://[^ )"`,>]+' . --include='0*.machine.md' --include='10_*.machine.md' \
      | sed 's/[.,;:]*$//' | sort -u) \
  <(grep -ohE  'https?://[^ )"`,>]+' 11_references.machine.md \
      | sed 's/[.,;:]*$//' | sort -u)
```

The twin-drift check. A URL present in a human twin but absent from its machine root violates the rule that the machine root is authoritative. Expect empty output:

```
cd payload/docs/advanced
comm -13 \
  <(grep -rhoE 'https?://[^ )"`,>]+' . --include='*.machine.md' | sed 's/[.,;:]*$//' | sort -u)
  <(grep -rhoE 'https?://[^ )"`,>]+' . --include='*.md' --exclude='*.machine.md' | sed 's/[.,;:
```

Before a publish or release, fetch every URL in the five source sections above and follow the redirects. If a final canonical URL differs from the one listed, update both this document and the citing documents, then re-render the affected PDFs with `/folio`. Record the new verification date in the header of the machine root.